Лабораторная работа № 14. Программирование в командном процессоре ОС UNIX. Расширенное программирование

НПИбд-03-25

Чжу Ино

2026-10-15

Содержание

1 Цель работы **2**

2 Задачи **2**

3 Теоретические сведения **2**
3.1 Механизм семафоров 2
3.2 Команда man 3
3.3 Переменная $RANDOM 3
3.4 Синтаксические ошибки 3
3.5 Конкатенация строк 3
3.6 Команда seq 3
3.7 Арифметика 3
3.8 Отличия zsh от bash 4

4 Выполнение лабораторной работы **4**
4.1 Скрипт 1: семафор 4
4.2 Скрипт 2: man 6
4.3 Скрипт 3: случайная строка 8

5 Выводы **10**

6 Контрольные вопросы **10**

7 Список литературы **11**

Список Иллюстраций

1 Скрипт семафор 5
2 chmod 6
3 Запуск 6
4 Два процесса 6
5 Вывод в tty 7
6 Скрипт man 7
7 chmod 8
8 ls 8
9 Не найдено 8
10 Скрипт random 9
11 chmod 9
12 10 9
13 20 10
14 Несколько 10

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задачи

1. Написать командный файл, реализующий упрощённый механизм семафоров.
2. Реализовать команду `man` с помощью командного файла.
3. Используя встроенную переменную `$RANDOM`, написать командный файл, генерирующий случайную последовательность букв латинского алфавита.

3 Теоретические сведения

3.1 Механизм семафоров

Семафор — примитив синхронизации, ограничивающий доступ к общему ресурсу. Простейшая реализация — файл-замок:

```
while [ -f "$SEMFILE" ]; do
    sleep 1
done
touch "$SEMFILE"
```

3.2 Команда man

Справка хранится в `/usr/share/man/man1/`. Файлы сжаты в `.gz`.

```
zcat /usr/share/man/man1/ls.1.gz | less
```

3.3 Переменная $RANDOM

Генерирует случайное число от 0 до 32767.

```
echo $RANDOM
```

3.4 Синтаксические ошибки

```
while [$1 != "exit"]    # ошибка: нет пробелов
while [ $1 != "exit" ]  # правильно
```

3.5 Конкатенация строк

```
s1="Hello"
s2="World"
s3="$s1 $s2"
```

3.6 Команда seq

Генерирует последовательность чисел:

```
seq 1 10
```

3.7 Арифметика

- `$((10/3))` — целочисленное деление, результат 3
- `for ((a=1; a <= LIMIT; a++))` — синтаксис C-подобного цикла, верен

3.8 Отличия zsh от bash

- zsh имеет более мощное автодополнение
- zsh поддерживает расширенные глоб-паттерны
- zsh более строгий к синтаксису

4 Выполнение лабораторной работы

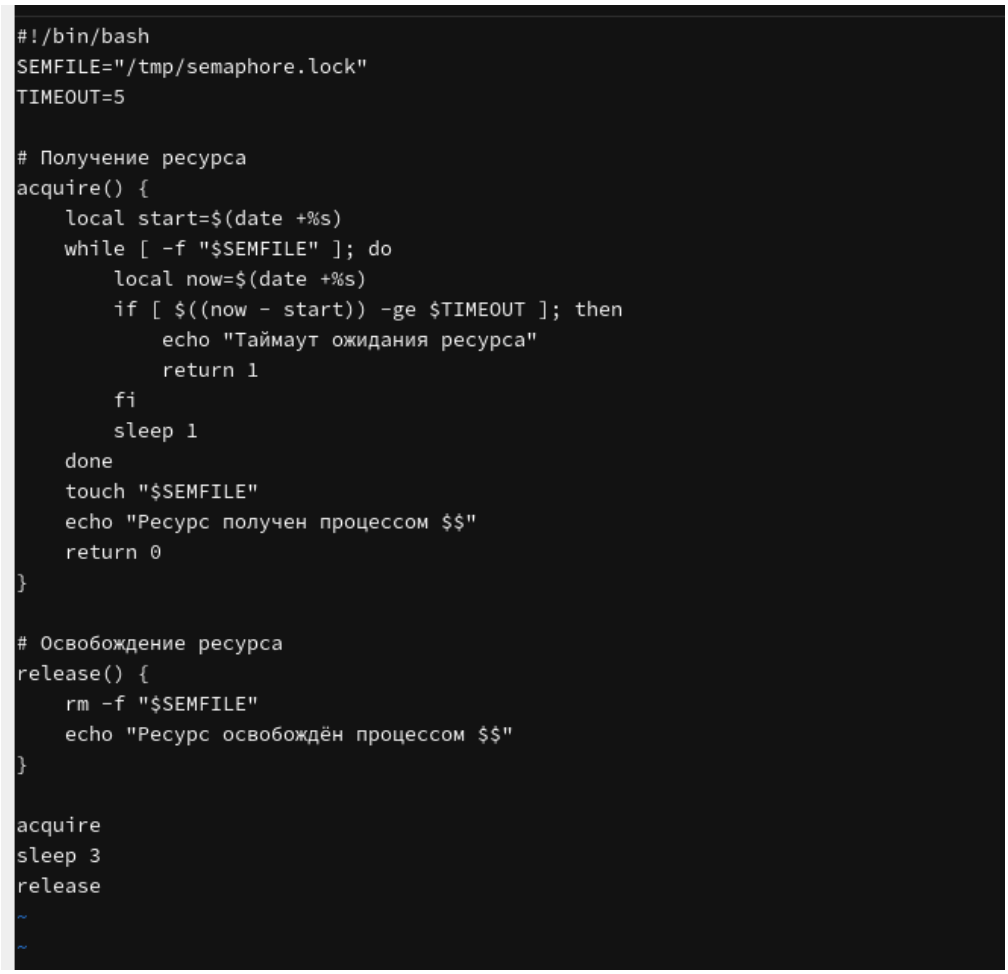
4.1 Скрипт 1: семафор

```
#!/bin/bash
SEMFILE="/tmp/semaphore.lock"
TIMEOUT=5

acquire() {
    local start=$(date +%s)
    while [ -f "$SEMFILE" ]; do
        local now=$(date +%s)
        if [ $((now - start)) -ge $TIMEOUT ]; then
            echo "Таймаут ожидания ресурса"
            return 1
        fi
        sleep 1
    done
    touch "$SEMFILE"
    echo "Ресурс получен процессом $$"
    return 0
}

release() {
    rm -f "$SEMFILE"
    echo "Ресурс освобождён процессом $$"
}

acquire
sleep 3
release
```



```
#!/bin/bash
SEMFILE="/tmp/semaphore.lock"
TIMEOUT=5

# Получение ресурса
acquire() {
    local start=$(date +%s)
    while [ -f "$SEMFILE" ]; do
        local now=$(date +%s)
        if [ $((now - start)) -ge $TIMEOUT ]; then
            echo "Таймаут ожидания ресурса"
            return 1
        fi
        sleep 1
    done
    touch "$SEMFILE"
    echo "Ресурс получен процессом $$"
    return 0
}

# Освобождение ресурса
release() {
    rm -f "$SEMFILE"
    echo "Ресурс освобождён процессом $$"
}

acquire
sleep 3
release
~
~
```

Рисунок 1: Скрипт семафор

```
chmod +x semaphore.sh
ls -l semaphore.sh
```

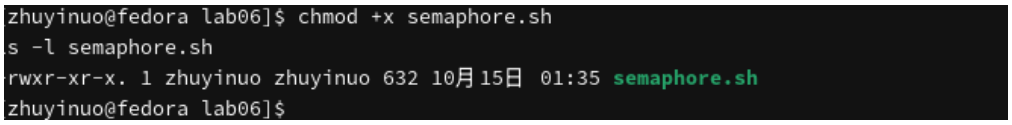


Рисунок 2: chmod

```
./semaphore.sh
```

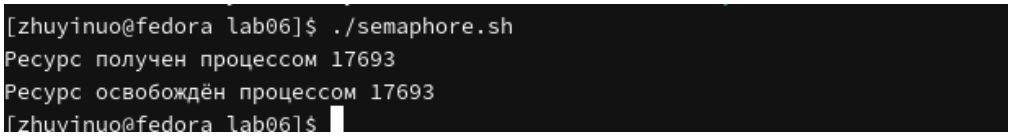


Рисунок 3: Запуск

```
./semaphore.sh &
./semaphore.sh &
wait
```

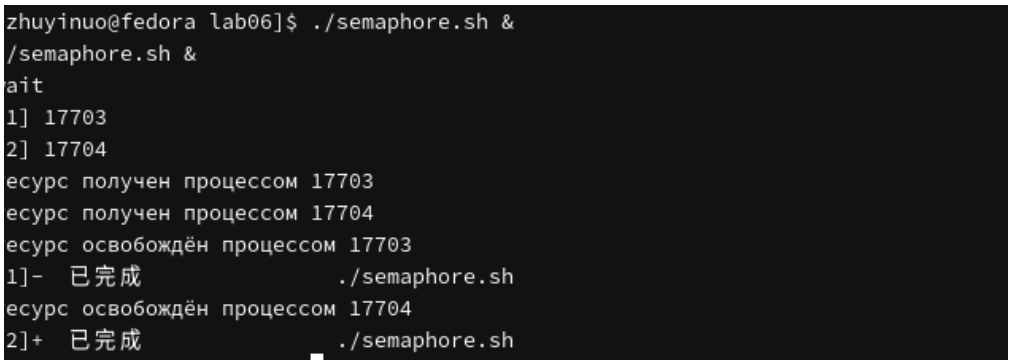


Рисунок 4: Два процесса

```
./semaphore.sh > /dev/tty2 &
./semaphore.sh > /dev/tty2 &
```

4.2 Скрипт 2: man

```
#!/bin/bash
MAN1="/usr/share/man/man1"
CMD=$1

if [ -z "$CMD" ]; then
```

```
[zhuyinuo@fedora lab06]$ ./semaphore.sh > /dev/tty2 &
./semaphore.sh > /dev/tty2 &
bash: /dev/tty2: 权限不够
[1] 17714
[2] 17715
```

Рисунок 5: Вывод в tty

```
    echo "Укажите имя команды"
    exit 1
fi

if [ -f "$MAN1/$CMD.1.gz" ]; then
    zcat "$MAN1/$CMD.1.gz" | less
elif [ -f "$MAN1/$CMD.1" ]; then
    less "$MAN1/$CMD.1"
else
   MANPAGE=$(find /usr/share/man -name "$CMD.*" -print -quit 2>/dev/null)
    if [ -n "$MANPAGE" ]; then
        man "$CMD" | less
    else
        echo "Справка для команды $CMD не найдена"
    fi
fi
```

```
#!/bin/bash
MAN1="/usr/share/man/man1"
CMD=$1

if [ -z "$CMD" ]; then
    echo "Укажите имя команды"
    exit 1
fi

# Поиск справки
if [ -f "$MAN1/$CMD.1.gz" ]; then
    zcat "$MAN1/$CMD.1.gz" | less
elif [ -f "$MAN1/$CMD.1" ]; then
    less "$MAN1/$CMD.1"
else
    # Поиск по всем man-каталогам
    MANPAGE=$(find /usr/share/man -name "$CMD.*" -print -quit 2>/dev/null)
    if [ -n "$MANPAGE" ]; then
        man "$CMD" | less
    else
        echo "Справка для команды $CMD не найдена"
    fi
fi
~
~
```

Рисунок 6: Скрипт man

```
chmod +x man_helper.sh
ls -l man_helper.sh
```

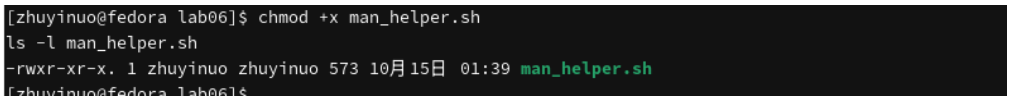


Рисунок 7: chmod

```
./man_helper.sh ls
```

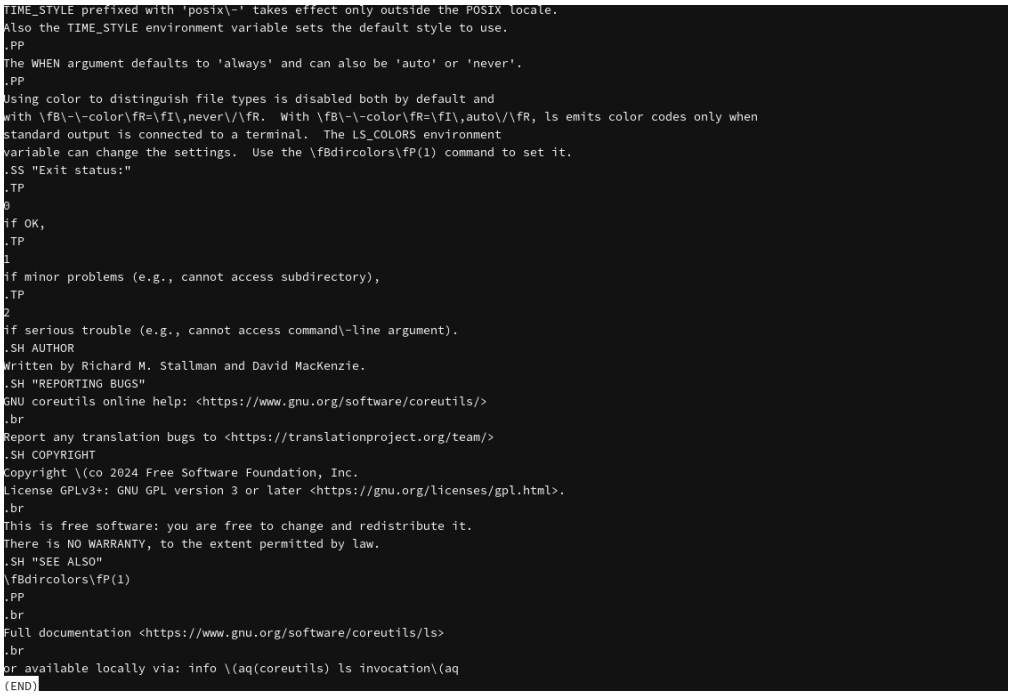


Рисунок 8: ls

```
./man_helper.sh nonexistentcmd
```

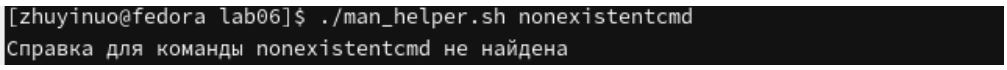


Рисунок 9: Не найдено

4.3 Скрипт 3: случайная строка

```
#!/bin/bash
LENGTH=${1:-10}

for i in $(seq 1 $LENGTH); do
```

```
    R=$((RANDOM % 52))
    if [ $R -lt 26 ]; then
        CODE=$((65 + R))
    else
        CODE=$((97 + R - 26))
    fi
    printf "\\$(printf '%03o' $CODE)"
done
echo
```

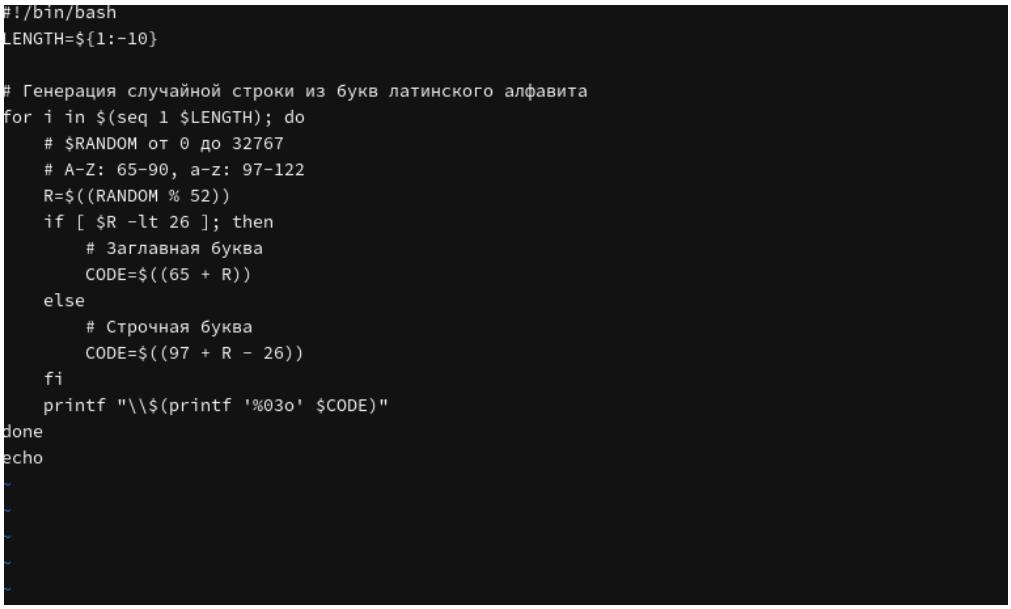


Рисунок 10: Скрипт random

```
chmod +x random_string.sh
ls -l random_string.sh
```

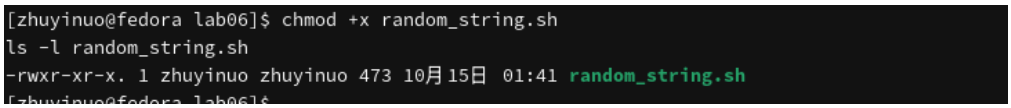


Рисунок 11: chmod

```
./random_string.sh 10
```

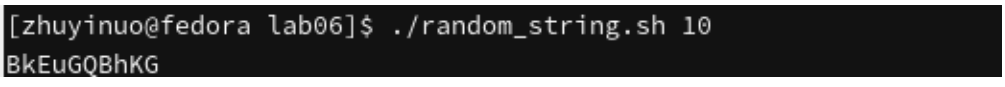


Рисунок 12: 10

```
./random_string.sh 20
```

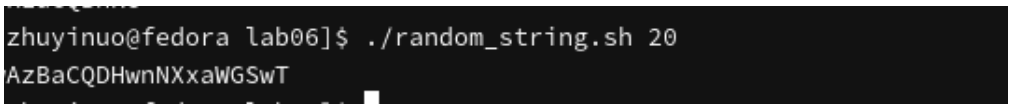


Рисунок 13: 20

```
for i in 1 2 3; do ./random_string.sh 8; done
```

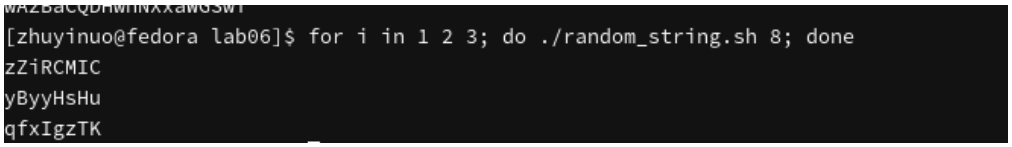


Рисунок 14: Несколько

5 Выводы

В ходе работы изучены расширенные возможности программирования в bash: механизм семафоров с файлом-замком, реализация команды `man` через `/usr/share/man`, генерация случайных строк с помощью `$RANDOM`.

6 Контрольные вопросы

1. Синтаксическая ошибка в строке.

```
while [$1 != "exit"]
```

Отсутствуют пробелы после `[` и перед `]`. Правильно:

```
while [ $1 != "exit" ]
```

2. Конкатенация строк.

```
s3="$s1 $s2"
```

3. Утилита seq.

Генерирует последовательность чисел. Альтернативы: `for i in {1..10}`, `while`.

4. Результат `$((10/3))`.

3 (целочисленное деление).

5. Отличия zsh от bash.

zsh имеет более мощное автодополнение, расширенные глоб-паттерны, более строгий синтаксис.

6. Синтаксис `for ((a=1; a <= LIMIT; a++))`.

Верен, это C-подобный цикл в bash.

7. Сравнение bash с другими языками.

Преимущества: простота, интеграция с ОС, работа с файлами. Недостатки: медленнее, слабая типизация, ограниченные структуры данных.

7 Список литературы

1. Курячий К. В. Операционная система UNIX. — М. : ИНТУИТ, 2016.
2. Робачевский А., Немнюгин С., Стесик О. Операционная система UNIX. — БХВ-Петербург, 2010.
3. Колисниченко Д. Н. Самоучитель системного администратора Linux. — БХВ-Петербург, 2011.